

Lec 18 Part 1 $\Rightarrow$ 30.1.

Lec_18

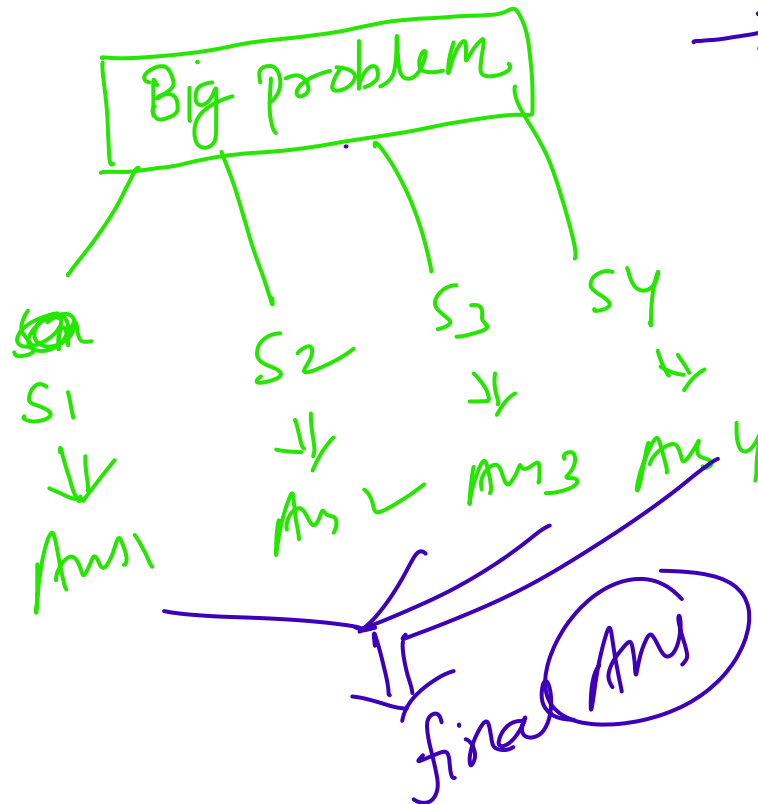
30.1.

DAC

Introduction to Divide and Conquer Algorithm

Basic of ~~DAC~~

Small problems
Solve $\Rightarrow$



$\rightarrow$ merge sort
 $\Downarrow$
Example of DAC

Divide and Conquer Algorithm is a problem-solving technique used to solve problems by dividing the main problem into subproblems, solving them individually and then merging them to find solution to the original problem.

Example:

Solving:

23, 6, 5, 10, 1, 13

1, 5, 6, 10, 13, 23

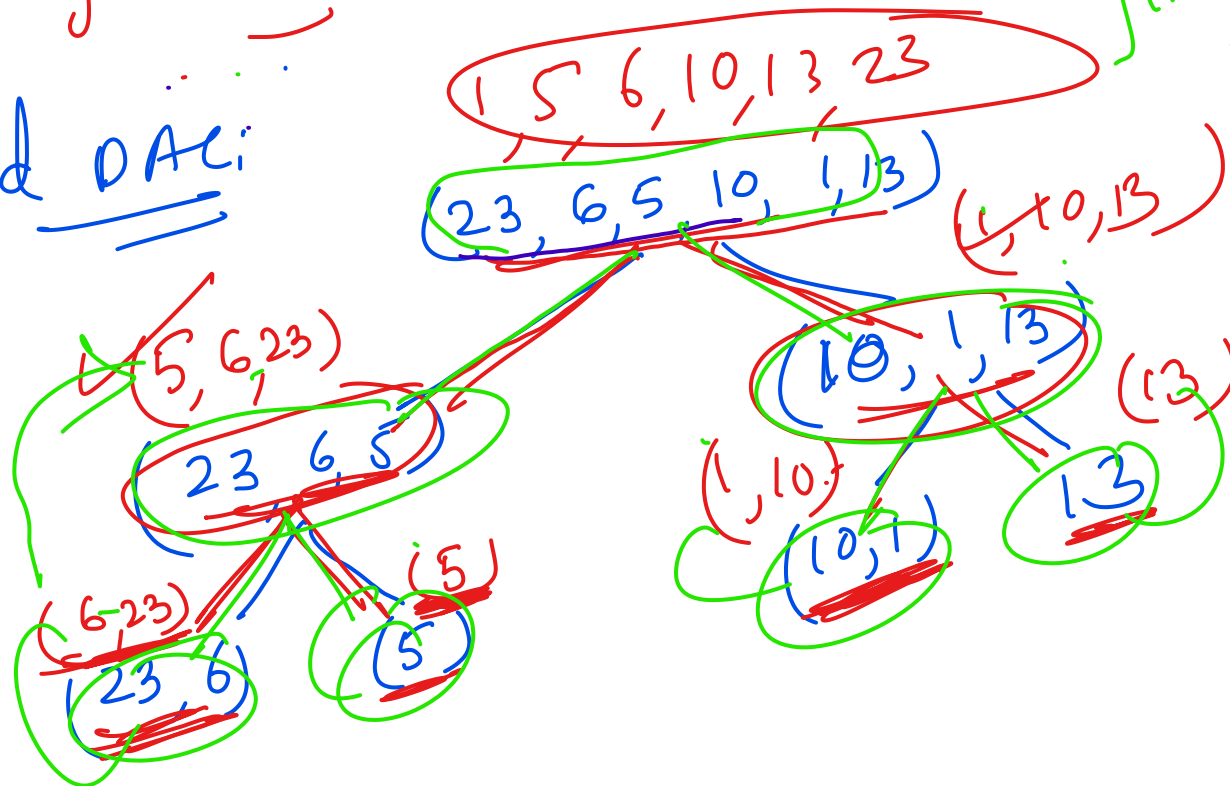
final solution

sort in increasing order:

lets understand DAC:

a, b

DAC



DAC

Basics of Divide and Conquer

algorithm

- Recursion
- Recurrence Relation
- Solving the Recurrence Relation

1) Substitution Method

2) Recursive Tree method

3) Master's Theorem

give

AA
PA

1-2 marks

100

Recursion

The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function.

Example: Factorial

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

```
int fact(n)
{
    for(int i=n; i>=1; i--)
    {
        ans = ans * i
    }
}
```

T.C

Comment

$$5 \times 4 \times 3 \times 2 \times 1$$

$$n \times n-1 \times n-2 \times \dots \times 1$$

factorial

include <iostream>
using namespace std;

int main

{ int n;

cin >> n;

int Ans = fact(n)

cout << Ans;

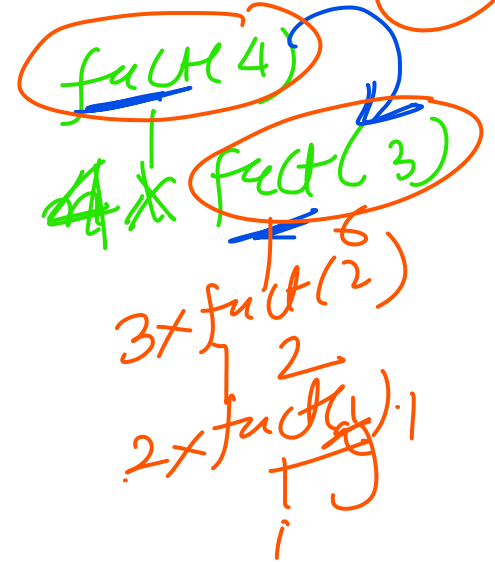
return 0; fn: n=4

Ans = fact(4)

}

int fact(n) ↗ recursive
{ if (n <= 1)
return 1;
return n * fact(n-1) ;

24



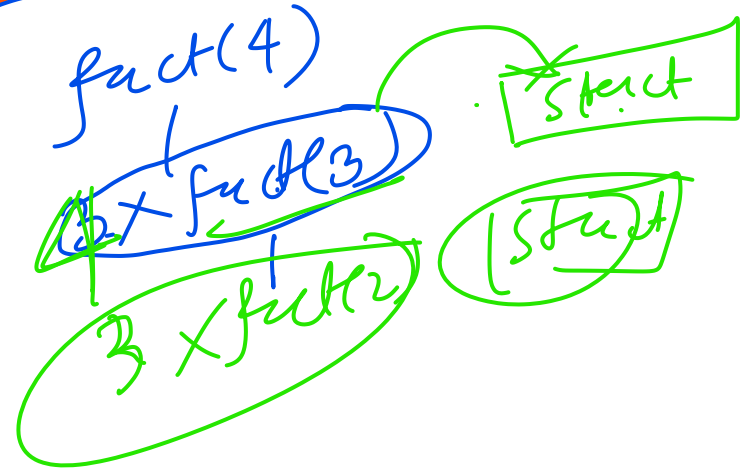
here fact(n) is calling fact(n-1) this is recursive
[example of]

passing
value can
be changed
But
same code
is

int fun(n, 4)
if (n = 0)
return 4 + 4;
else
return 1 + fun(n-1, 4-2);

termination
condition

executed



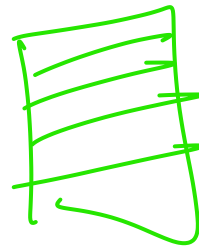
Properties of a Recursion

- ✓ To execute recursive program, we use stack data Structure
- ✓ Termination Condition:

- While 1 R.P. running, we are using one stack only, but many stack units
- Recursion is nothing but solving bigger problem interims of smaller problems
- While R.P. is running no. of stack units used is known as depth of the depth

recursion

fact(4)
→ fact(3)
→ fact(2)
→ fact(1)



fact(4)
fact(3)
fact(2)
fact(1)

4 → depth

Ex.

int func(a, b)

{ if(a=0)
return 2;

if(b=0)
return 3;

else return

func(a-2, b-3)

Beautiful
recursive

Thanky

